

Documento de Arquitetura de Software (DAS) - CoMangá

1. Introdução

O projeto CoMangá surge como uma solução para o gerenciamento e catalogação de coleções físicas de mangás. O sistema busca reduzir a complexidade causada por múltiplas obras, edições, editoras e volumes, oferecendo ao usuário uma estante digital organizada, além de recursos de catálogo, checklist e lista de desejos.

O propósito deste documento é registrar as principais decisões arquiteturais que sustentam a plataforma, garantindo integridade dos dados, segurança, escalabilidade para o MVP e alinhamento entre requisitos, modelagem e implementação.

2. Representação Arquitetural

A arquitetura do CoMangá segue o paradigma cliente-servidor, organizada em três camadas: Apresentação, Aplicação e Persistência. O sistema utiliza a stack PERN, composta por PostgreSQL, Express, React e Node.js, com TypeScript adotado no frontend e no backend para aumentar a segurança e a manutenibilidade do código.

2.1. Camada de Apresentação (Frontend)

A camada de apresentação é construída como uma Single Page Application (SPA), utilizando React.js com TypeScript e Vite. Essa camada é responsável pela interface do usuário, navegação, renderização das páginas, controle de estados locais e globais, validações de formulário e consumo das rotas da API.

O frontend é hospedado na Vercel, aproveitando deploy automatizado, distribuição via CDN e certificados SSL automáticos.

2.2. Camada de Aplicação (Backend)

A camada de aplicação utiliza Node.js com Express.js e TypeScript. Ela expõe uma API RESTful responsável por centralizar as regras de negócio do sistema, como cadastro, ativação de conta, autenticação, controle de sessão, consulta de perfil, atualização de preferência +18 e futuras regras de catálogo, estante, checklist e lista de desejos.

O backend é hospedado na Render como um serviço web persistente. Por utilizar TypeScript, o código da aplicação passa por uma etapa de build, na qual os arquivos `.ts` são compilados para JavaScript na pasta `dist`, que é executada em produção pelo Node.js.

2.3. Camada de Persistência (Banco de Dados)

A camada de persistência utiliza PostgreSQL, hospedado em nuvem por meio do Neon DB.

O PostgreSQL foi escolhido por oferecer forte integridade referencial, suporte a relacionamentos, constraints, transações e modelagem relacional adequada à hierarquia do domínio do CoMangá.

O modelo de dados prevê a hierarquia Obra Matriz → Edição → Volume, além de entidades relacionadas a usuários, sessões, permissões, estante digital, checklist e lista de desejos.

O acesso ao banco é realizado por meio do Prisma ORM, que fornece mapeamento entre os modelos da aplicação e as tabelas relacionais do PostgreSQL. As alterações estruturais do banco são controladas por migrations Prisma, permitindo versionamento e reprodutibilidade do schema.

3. Segurança e Controle de Sessão

O CoMangá utiliza autenticação baseada em sessão stateful. Após o login, o backend gera um token opaco de sessão, armazena seu hash na tabela `sessions` e envia o token ao navegador em um cookie HttpOnly.

Essa abordagem evita o armazenamento de tokens sensíveis em localStorage e permite revogação explícita de sessão no logout. Rotas protegidas validam a sessão ativa no banco antes de permitir o acesso.

As principais decisões de segurança incluem:

- Cookie HttpOnly para reduzir exposição a ataques XSS;
- Controle de sessão no banco de dados;
- Revogação de sessão no logout;
- Validação de permissões por nível de acesso;
- Uso de variáveis de ambiente para segredos;
- Controle de CORS para permitir apenas origens autorizadas;
- Criptografia de senha com bcrypt;
- Tokens de ativação com expiração temporal.

4. Qualidade, Testes e Integração Contínua

O projeto utiliza testes automatizados e ferramentas de qualidade para reduzir regressões.

No backend, são utilizados Jest e Supertest para testes unitários e de integração das rotas da API, incluindo cadastro, ativação, login, logout, consulta de perfil e atualização de preferência +18.

No frontend, são utilizados Vitest e React Testing Library para testar componentes, contexto de autenticação e interações relevantes da interface.

O ESLint é utilizado para análise estática no frontend e no backend. O GitHub Actions executa uma pipeline de qualidade com instalação de dependências, aplicação de migrations no banco de teste, build TypeScript, testes automatizados e lint.

5. Ambientes e Deploy

O projeto utiliza ambientes separados para desenvolvimento local, testes automatizados e homologação.

No ambiente de homologação:

- O frontend é hospedado na Vercel;
- O backend é hospedado na Render;
- O banco PostgreSQL é hospedado no Neon DB;
- As variáveis de ambiente configuram conexões, CORS, cookies, SMTP e banco de dados;
- O deploy é realizado de forma automatizada pelas plataformas a partir das branches configuradas.

O backend utiliza o seguinte fluxo de build em homologação:

```
npm ci --include=dev
npm run prisma:generate
npm run migrate
npm run build
```

A execução em produção é feita por:

```
npm start
```

6. Metas e Restrições Arquiteturais

Custo Zero (MVP)

O sistema deve operar em camadas gratuitas dos provedores utilizados, como Vercel, Render e Neon DB.

Integridade Referencial

É mandatório o uso de chaves primárias, chaves estrangeiras, constraints e migrations para evitar registros órfãos, duplicidades indevidas e inconsistência de dados.

Segurança e Visibilidade

O sistema deve diferenciar usuários comuns e administradores, além de controlar permissões e visibilidade de recursos sensíveis.

Manutenibilidade

O uso de TypeScript, Prisma ORM, testes automatizados e organização em camadas deve facilitar evolução, refatoração e implementação de novas funcionalidades no PMI IV.

Escalabilidade do MVP

A arquitetura deve suportar a evolução gradual do catálogo, estante digital, checklist, lista de desejos e módulos administrativos sem exigir reestruturação completa.